

VAE vs. DDPM for Anime Face Generation

From-scratch PyTorch implementation and comparison — kaggle splcher/animefacedataset, 64×64
Code: github.com/minanasser54/DDPM_vs_VAE — Checkpoints: HF Hub

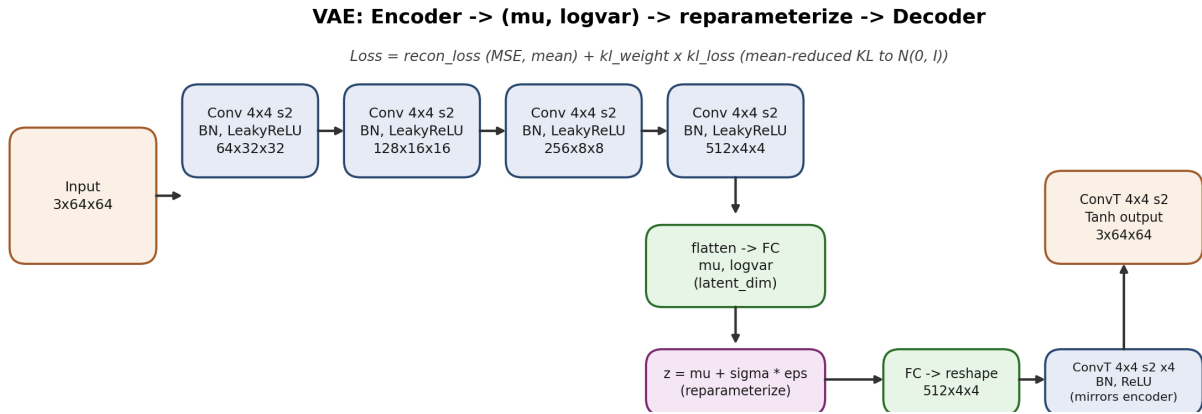
1 Implementation & Architecture

Both models are plain PyTorch nn.Modules with hand-written training loops (train_vae.py, train_ddpm.py) no diffusers. One script per responsibility (data.py, vae.py, ddpm.py, eval_*.py, benchmark.py), all hyperparameters centralized in conf.py.

1.1 VAE (vae.py)

Conv encoder (64 → 32 → 16 → 8 → 4, base 64 channels doubling each stage, BN+LeakyReLU)

→ FC heads for $\mu, \log \sigma^2$ → reparameterize → FC+reshape → mirrored transposed-conv decoder, tanh output.

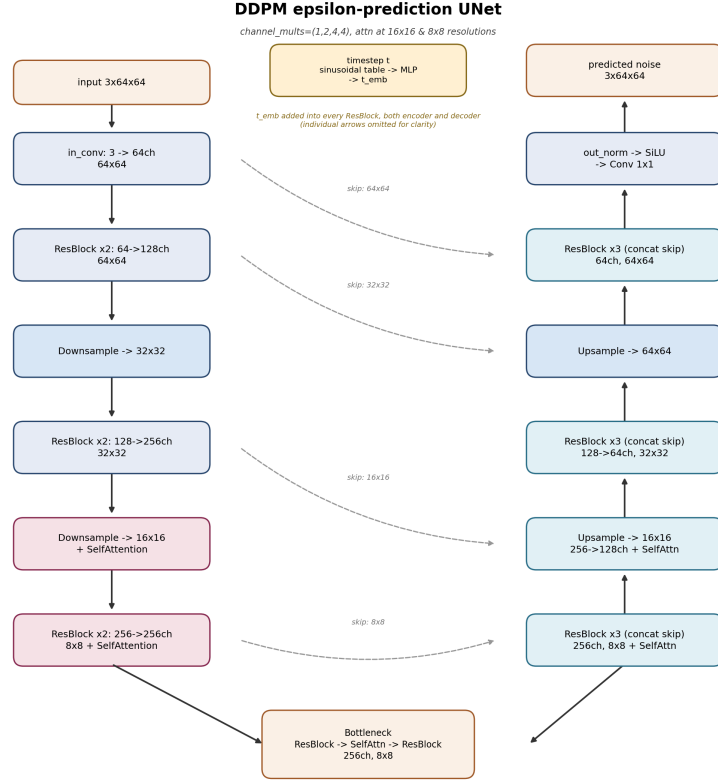


$$z = \mu + \sigma \cdot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I) \quad \mathcal{L}_{\text{VAE}} = \underbrace{\text{MSE}(\hat{x}, x)}_{\mathcal{L}_{\text{recon}}} + \beta \cdot \underbrace{-\frac{1}{2} \text{mean}(1 + \log \sigma^2 - \mu^2 - \sigma^2)}_{\mathcal{L}_{\text{KL}}}$$

Both terms are **mean**-reduced (not summed), which keeps them at comparable, dimension-independent scale so β (VAE_KL_WEIGHT) is a meaningful, tunable knob between reconstruction and generation rather than an arbitrary constant.

1.2 DDPM (ddpm.py)

Ho et al. (2020) formulation: fixed forward noising process, learned $\varepsilon_{\theta}(x_t, t)$ UNet. Pre-activation ResBlocks (GroupNorm→SiLU→Conv ×2, residual skip conv on channel change) with timestep embedding injected into **every** block; self-attention only at 16×16/8×8; sinusoidal timestep table → learned MLP; standard U-shaped encoder/bottleneck/decoder with concatenated skips.



$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon \quad \mathcal{L}_{\text{DDPM}} = \mathbb{E}_{x_0, t, \varepsilon} [\|\varepsilon - \varepsilon_{\theta}(x_t, t)\|^2]$$

- Linear beta schedule ($10^{-4} \rightarrow 0.02$, $T = 1000$).
- Ancestral reverse sampling (Algorithm 2, Ho et al.), $x_T \sim \mathcal{N}(0, I) \rightarrow \dots \rightarrow x_0$.
- Training stability additions: **gradient clipping** (0.7) and **EMA** of weights (decay 0.9995, warmup before averaging starts) all sampling/eval below uses EMA weights.
- **Gradient accumulation** (--grad_accum_steps) allows a larger effective batch size than fits physically in memory.

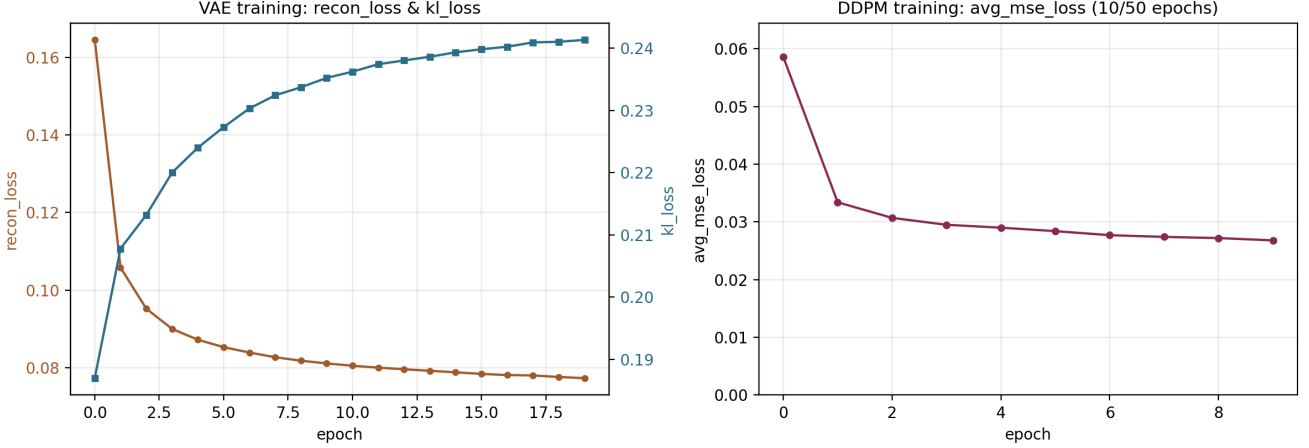
2 Experimental Setup

Hyperparameter	VAE	DDPM
Base channels	64	64
Latent dim.	256	—
Physical batch size	256	32
Grad. accum. steps	—	2 (effective batch 64)
Epochs configured / completed	20 / 20	50 / 10 (session-limited)
Learning rate	2×10^{-4}	2×10^{-4}
KL weight β	0.1	—
Diffusion steps T / schedule	—	1000 / linear
Grad clip / EMA decay	—	0.7 / 0.9995

Caveat: DDPM trained only **10/50 configured epochs** (Colab session limits); avg_mse_loss was still falling with no plateau ($0.0586 \rightarrow 0.0268$). All DDPM results below describe a partially-trained model. Evaluation: generate from pure noise/prior (not reconstructions) vs. held-out real test split ($\approx 1,900$ imgs); metrics = **FID** (pytorch-fid, InceptionV3-pool3) + **Inception Score** (torchmetrics).

3 Results

3.1 Training curves



- **VAE:** recon_loss falls steadily ($0.165 \rightarrow 0.077$); kl_loss rises from near-zero and **stabilizes** nonzero (≈ 0.24) signature of healthy training (latent space is actually used).
- **DDPM:** avg_mse_loss drops sharply first 2 epochs ($0.0586 \rightarrow 0.0307$), then diminishing but continued improvement ($\rightarrow 0.0268$) consistent with coarse structure being learned first, fine detail later.

3.2 Quantitative benchmark

Model	FID ($\downarrow$)	Inception Score ($\uparrow$)	Training extent
VAE	282.93	2.575 ± 0.475	20/20 epochs
DDPM	91.97	2.168 ± 0.295	10/50 epochs

DDPM’s FID is $\approx 3\times$ better despite $\frac{1}{5}$ the completed epochs, consistent with diffusion models’ known tendency to avoid the blur that a pixel-wise-MSE VAE decoder structurally produces. Inception Score is close and low for both expected, since IS was designed around ImageNet’s 1000-class diversity and anime faces are visually one narrow class.

3.3 Samples

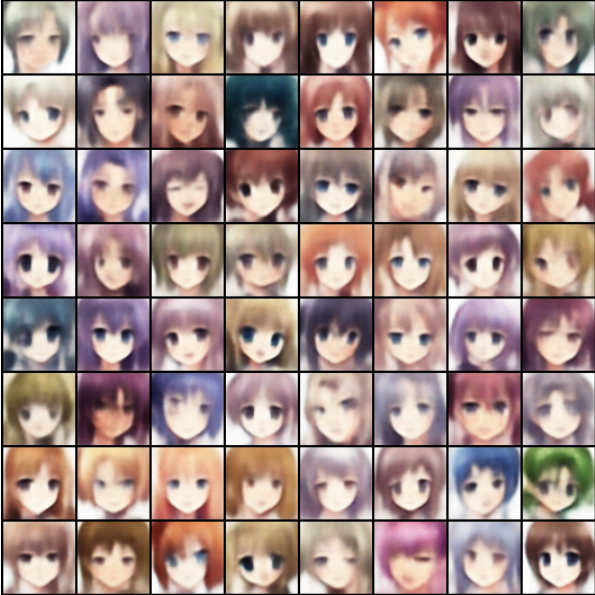


Figure 4: VAE: 64 prior samples, $z \sim \mathcal{N}(0, I)$.



Figure 5: DDPM: 16 samples, full 1000-step reverse loop, EMA weights.

- VAE samples: consistent but soft/blurry, low-contrast, smoothed hair strand detail, direct consequence of a pixel-wise averaging decoder objective. Broad hair/eye-color diversity, no mode collapse.
- DDPM samples: visibly sharper, defined eye highlights and hair texture on successful draws, matching the FID gap but **2/16 blank** and **11/16 near-noise**, i.e. occasional outright reverse-diffusion failures at this training stage (undertrained model, not an architectural flaw).

Axis	VAE	DDPM (10/50 ep.)
Realism	Soft/blurry, low detail contrast	Sharper linework on success matches FID
Diversity	High, no mode collapse	High on successful samples
Consistency	Very high (~64/64 usable)	Lower (~2/16 failed outright)

4 Conclusion

Core difference is **how each model maps noise to an image**:

- **VAE**: single deterministic decoder $x = \text{Dec}(z)$ trained to minimize **average** pixel error over a multi-modal distribution $\rightarrow$ structurally pushed toward safe, smoothed outputs that hedge across plausible variations rather than committing to one sharp realization.
- **DDPM**: learns to **iteratively** undo a fixed noising process over T small steps; each step’s target (predict the added noise) is a simpler, better-posed problem than “produce a whole image in one shot,” letting fine detail accumulate gradually rather than being averaged away up front at the cost of T sequential network evaluations per sample (vs. one decoder pass).

Required info: full code, `conf.py`, checkpoints, and reproduction steps are in the linked GitHub repo and Hugging Face Hub (see header); `README.md` documents exact commands (`main.py`, per-stage scripts, Colab/Kaggle notes, `--resume` for session-limited training).